

CMPE 220 — System Software

Spring 2026

Assignment 1

Software CPU Design and Implementation

Team members

Faiq Malik

Date: May 15, 2026

GitHub Repository

The complete source code, any built artifacts or example binaries included in the repository, demonstration video, and documentation are maintained at:

`https://github.com/Faiq2001/software\_cpu\_design-main`

The repository root contains `cpu16_core.cpp` (single-file CPU, assembler, and emulator), demonstration media under `demos/` (including `demos/Demo.mp4`), diagrams under `figures/` when present, any prebuilt program images (for example `fib.bin`), and this report under `reports/`.

Download, Compile, and Run

Download

Clone the repository with Git, or download a ZIP archive from the GitHub page above and extract it to a working directory on macOS or Linux.

Requirements

- C++ compiler with C++17 support (g++ recommended).
- Terminal access.

Compile

From the repository root directory:

```
g++ -std=c++17 -O2 -o cpu16 cpu16_core.cpp
```

This produces the executable `cpu16`, which implements assembler mode (`asm`), bare-memory emulator mode (`emu`), and assemble-and-run mode (`run`).

Run (examples)

Typical commands when example assembly sources are under `examples/`:

```
./cpu16 run examples/hello.asm
```

```
./cpu16 run examples/timer.asm
```

```
./cpu16 asm examples/fib.asm -o fib.bin
```

```
./cpu16 emu fib.bin --base 0x0000 --pc 0x0100 --dump 0x0100 0x0140
```

The Fibonacci listing uses `.org 0x0100`; when emulating `fib.bin`, set the program counter to `0x0100` as in the command above. If `fib.bin` lives in the repository root instead of beside the assembly file, use the same arguments with that path, for example:

```
./cpu16 emu fib.bin --base 0x0000 --pc 0x0100 --dump 0x0100 0x0140
```

To assemble factorial from source when `examples/fact.asm` is available, then load and dump memory:

```
./cpu16 asm examples/fact.asm -o fact.bin
```

```
./cpu16 emu fact.bin --base 0x0000 --pc 0x0000 --dump 0x0000 0x01FF
```

Video

The Fibonacci demonstration required for the assignment is included as `demos/Demo.mp4` in the repository.

Team Member Contributions

Faiq Malik

Contributions include:

- **Architecture and ISA:** 16-bit CPU organization (registers, ALU, control flow, memory map, MMIO), instruction formats and encodings, and flag semantics as implemented in `cpu16_core.cpp`.
- **Emulator:** Fetch–decode–execute, register file, ALU with condition flags, program counter, stack conventions, per-instruction timer step, and UART/timer MMIO behavior.
- **Assembler:** Two-pass assembly with labels, literals, directives (`.org`, `.word`, `.stringz`), and resolution of forward references.
- **Example programs:** Hello world, timer, Fibonacci, and factorial (or equivalent) assembly used for demonstration and grading.
- **Deliverables:** Repository layout, grader-facing build and run documentation (including this report), demonstration video (`demos/Demo.mp4`), supporting assets as required, and this report submitted as PDF for the course.